

Evaluation

A hands-on course in building agents by building their harness

Lesson 10

Every change is a bet

- Lesson 9's savings: a trim might cut the line that mattered, a summary might lose a fact
- Same for any change: a prompt line, a model, a step limit, a tool
- After the change the agent still runs and the trace still looks normal
- You find out the way you do for untested software: someone uses it, and it's wrong

Evaluation finds out first

Run the agent on tasks whose right outcome you already know, check what happened, and compare with before the change.

1. **Tasks with known outcomes**
2. **Grading**
3. **Comparing**

Grade the world, not the words

- A case is three things: a starting state, a task, a way to tell if the outcome is right
- The outcome is checked **without the agent**: a file, an exit code, the folder afterwards
- An agent that says "done" has proved nothing; a test that passes has
- Any route that ends in the right place passes, two steps or five
- Each case starts from a fresh folder, so cases can't affect each other

Grade by code where you can

- **By code:** a command whose exit status says pass or fail. Cheap, exact, the same verdict every time
- **By a model:** for what code can't read, like an explanation. A second model gets a rubric and the output
- A model grader has its own mistakes, so it's tested too, on answers whose grade you know

A grade you can't trust is worse than none, because you'll believe it.

A count means nothing alone

- The model isn't deterministic: run each case several times and count the passes
- 11 of 12 is good or bad only against something: run the same cases **before and after**
- Record steps, tokens and time too: a change can pass everything and cost twice as much
- Exit non-zero when something got worse, so a git hook or CI job runs it on **every** change

Built on the whole harness, from the outside

- It treats the agent as a box: a task goes in, the world is changed
- That runs all five primitives together, so it catches what only shows when they work together
- **Control flow:** a loop that sets up, runs, grades, records, stops
- The task goes in as **input**; the check runs as **output**; a model grader uses the **model interface**
- Lesson 7's trace says what happened; evaluation says whether it was right

Four cases, each checkable afterwards (abridged)

```
CASES = [  
    {"name": "count", "setup": "seq 1 37 > numbers.txt", "task": "How many lines are in numbers.txt? ...",  
     "check": "[ \"$(tr -d ' \\n' < answer.txt)\" = 37 ]"},  
    {"name": "fix", "setup": "printf ... > calc.py; printf ... > test.py", "task": "test.py fails. ...",  
     "check": "python3 test.py && grep -q 'add(2, 3)' test.py"},  
    {"name": "rename", "setup": "touch a.txt b.txt c.txt", "task": "Rename every .txt file ...",  
     "check": "[ -e a.md ] && [ -e b.md ] && [ -e c.md ] && ! ls *.txt 2>/dev/null"},  
    {"name": "remember", "setup": "true", "task": "Remember that I prefer short answers.",  
     "check": "grep -qi short .quark/memory/memory.md"},  
]
```

Run quark itself, in a fresh folder (abridged)

```
for case in cases:
    where = tempfile.mkdtemp(prefix=f"eval-{case['name']}-")
    subprocess.run(case["setup"], shell=True, cwd=where)
    start = time.time()
    try: subprocess.run([sys.executable, os.path.abspath(__file__), case["task"]], cwd=where, ...)
    except subprocess.TimeoutExpired: pass
    passed = subprocess.run(case["check"], shell=True, cwd=where, ...).returncode == 0
    seconds = round(time.time() - start, 1)
```

- Not a mock: the same file, prompt, guard, sandbox, retries and tracing
- `input="y\n" * 50` answers the guard's questions; its policy still blocks
- Over five minutes, it's killed and graded on what it left behind

The cost comes from the trace (abridged)

```

events = [json.loads(line) for line in open(f"{where}/.quark/traces.jsonl")] if ... else []
models = [e for e in events if e["event"] == "model"]
tokens = sum(e["input_tokens"] + e["output_tokens"] + e["cache_read"] + ... for e in models)
...
    if passed: shutil.rmtree(where, ignore_errors=True)
    else: failed += 1
print(f"{len(cases) - failed}/{len(cases)} passed")
return 1 if failed else 0

if sys.argv[1:2] == ["--eval"]: sys.exit(evaluate(sys.argv[2:]))

```

- Steps are the child run's `model` events; tokens include cache reads
- A failed case keeps its folder; a passed one is deleted
- One new section, caught before `# — input —` reads anything

It remembers last time

```
log, before = ".quark/evals.jsonl", {}  
os.makedirs(".quark", exist_ok=True)  
if os.path.exists(log):  
    for line in open(log): before[json.loads(line)["case"]] = json.loads(line)["passed"]  
cases, failed = [c for c in CASES if not names or c["name"] in names], 0
```

- A case that passed last time and fails now says **REGRESSED**
- Not "it failed", but "it used to pass"
- Run some cases by name: `quark.py --eval fix rename`

A baseline: what normal looks like

```
{"event": "stopped"}
```

The calculator still subtracts. The trace shows one model call, which asked for `cat calc.py test.py`, then the guardrail's `stopped`: the run hit the step limit before the model had seen what it read. The episode in the same folder has the request itself. Reading a failure like this is the most useful thing an evaluation gives you, which is why the folder is kept.

Going further

```
- 19,000 to 29,000 tokens for tiny tasks: the prompt is about eight thousand, read every step
- `fix` took three steps because it read the code before changing it

<!-- Within a case the later steps read the prompt from the cache. Each case is a new folder with its own path in the prompt, so one case's cache doesn't serve the next. -->

---

# A change that looks harmless

`MAX STEPS` from 20 to 1, to cap what a run can spend:
```